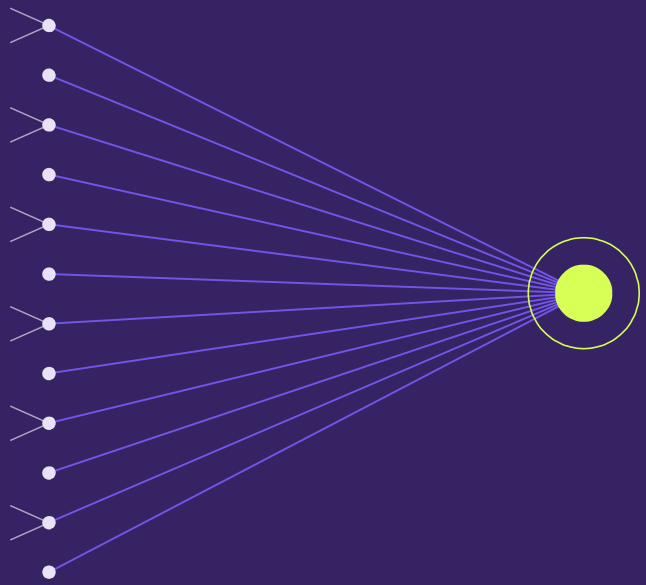
RANDOM. FOREST.

500 TREES. ONE PREDICTION.



Ensembles por Bagging: Random Forest

Objetivos da aula

01 Explicar o que é Ensemble Learning e quando ele ajuda

02 Construir amostras bootstrap com reposição

03 Calcular a probabilidade de uma amostra ser OOB

04 Diferenciar Bagging de Random Forest

05 Entender votação, média e probabilidades médias

06 Configurar os hiperparâmetros principais

07 Interpretar OOB e importância de características

08 Comparar uma árvore contra 500 com Scikit-learn

Roteiro de 120 minutos



10	15	20	20	15	30	10
Objetivos e árvores	Ensemble e Bagging	Bootstrap e OOB	Random Forest e características	Agregação, hiperparâmetros, importância	Uma árvore contra 500 árvores, do notebook vazio ao resultado	Quiz e desafio

Extensões do exercício (nº de árvores, profundidade, permutação) seguem no notebook se o tempo for insuficiente.

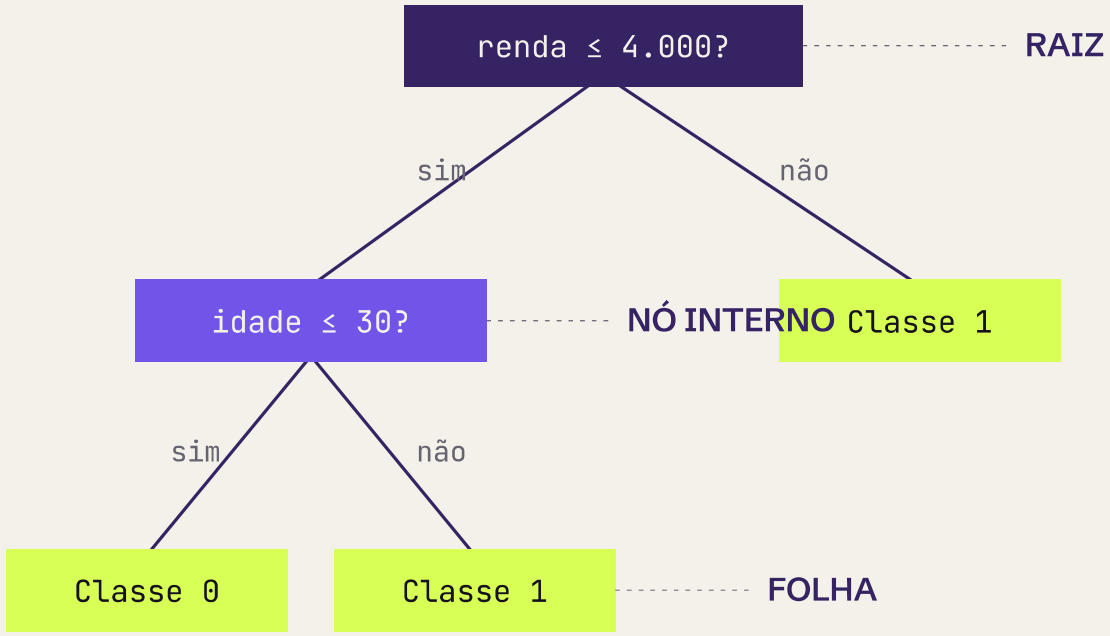
01

RECAPITULAÇÃO · AULA 6.9

Árvore de Decisão

Raiz, nós, folhas, Gini, profundidade e overfitting.

Anatomia de uma árvore



Raiz

Primeira pergunta, aplicada a todas as amostras.

Nó interno

Pergunta aplicada só às amostras que chegaram até ele.

Folha

Devolve a previsão: a classe majoritária do treino naquela região.

Profundidade

Nº de perguntas no caminho mais longo. Aqui: 2.

Valores ilustrativos.

Gini e a melhor divisão

$$G = 1 - \sum_k p_k^2$$

- G — impureza de Gini do nó (0 = nó puro)
- k — índice das classes
- p_k — proporção de amostras da classe k no nó

Melhor divisão = maior redução entre o Gini do nó pai e o Gini ponderado dos filhos.

Exemplo · nó com 10 amostras (6 A, 4 B)

Pai: $1 - (0{,}6^2 + 0{,}4^2) = 1 - 0{,}52 = \mathbf{0{,}48}$

Esq. (5 A, 0 B): $1 - (1^2 + 0^2) = \mathbf{0}$

Dir. (1 A, 4 B): $1 - (0{,}2^2 + 0{,}8^2) = \mathbf{0{,}32}$

Ponderado: $5/10 \cdot 0 + 5/10 \cdot 0{,}32 = \mathbf{0{,}16}$

Redução: **$0{,}48 - 0{,}16 = 0{,}32$**

Profundidade e critérios de parada

`max_depth`	Limita o nº de perguntas do caminho mais longo
`min_samples_split`	Mínimo de amostras para tentar dividir um nó
`min_samples_leaf`	Mínimo de amostras em cada folha
`nó puro`	Gini = 0: não há o que separar

Padrão do DecisionTreeClassifier: max_depth=None (sem limite).

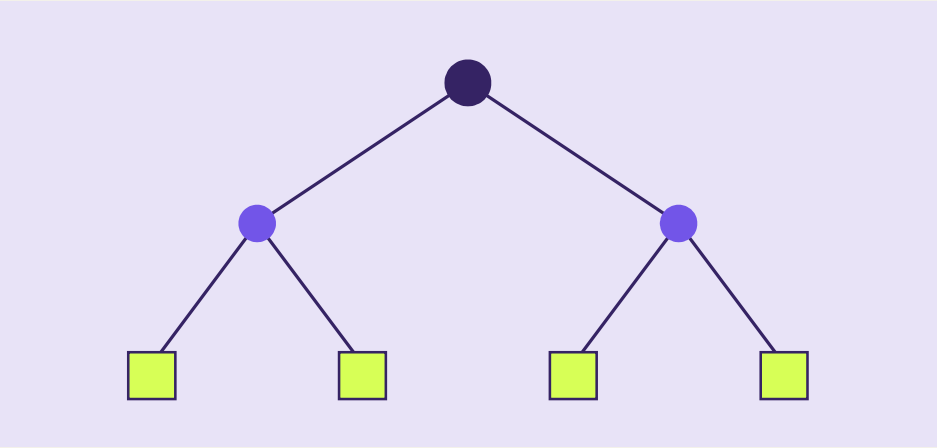
PERGUNTA

O que acontece quando uma única árvore aprende padrões específicos demais do treinamento?

Árvore rasa e árvore profunda

Rasa

`max_depth = 2`



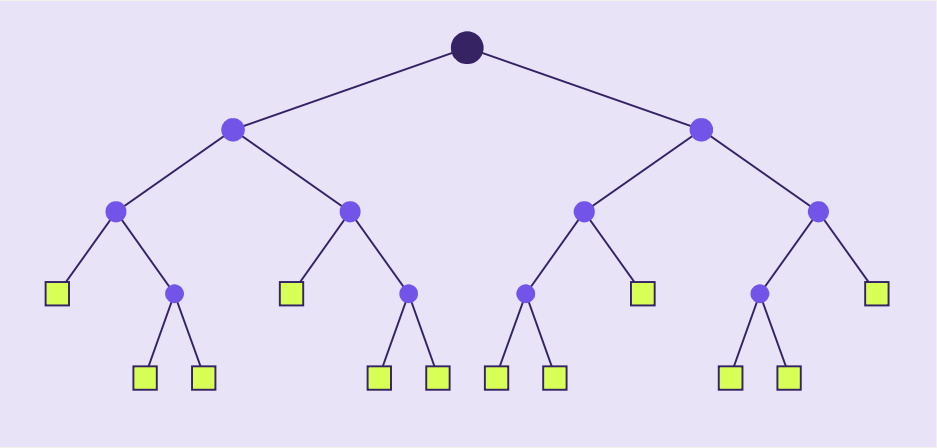
Poucas regras, fáceis de ler

Pode errar até no treino (alto viés)

Estrutura estável a pequenas mudanças

Profunda

`max_depth = None`



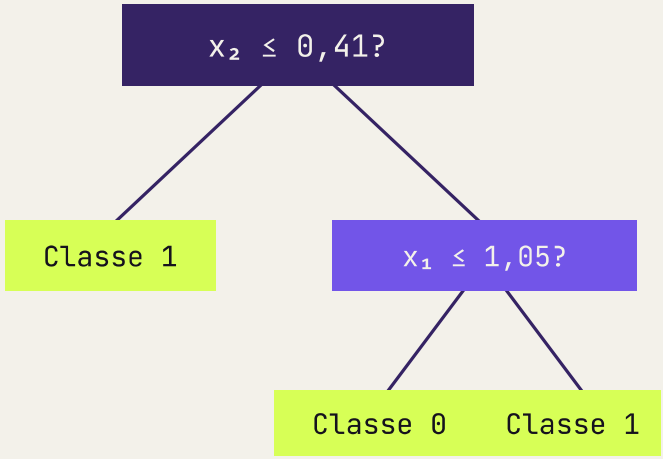
Folhas com poucas amostras

Treino quase perfeito, pode decorar ruído

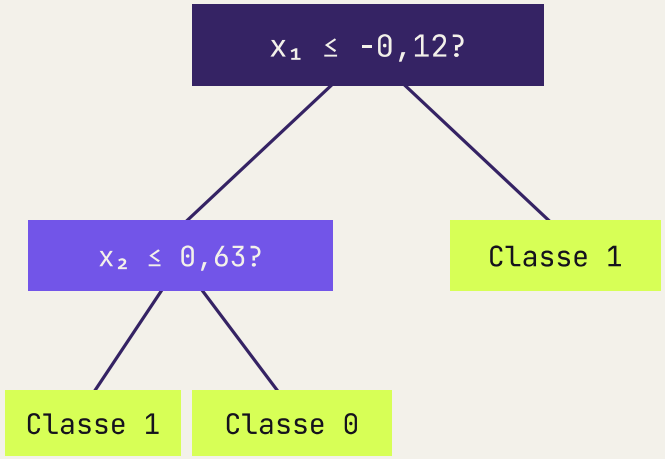
Alta variância: muda muito com os dados

Uma árvore é sensível às amostras

Amostra de treino A



Amostra de treino B



Sensibilidade: poucos pontos diferentes mudam a raiz

Estrutura: raiz nova muda tudo abaixo dela

Previsões: o mesmo ponto pode receber classes diferentes

Overfitting: cada árvore molda-se ao ruído da sua amostra

Limiares ilustrativos.

E se combinássemos as previsões de várias árvores?

Condição 1 · cada árvore é útil

Condição 2 · as árvores erram de formas diferentes

02

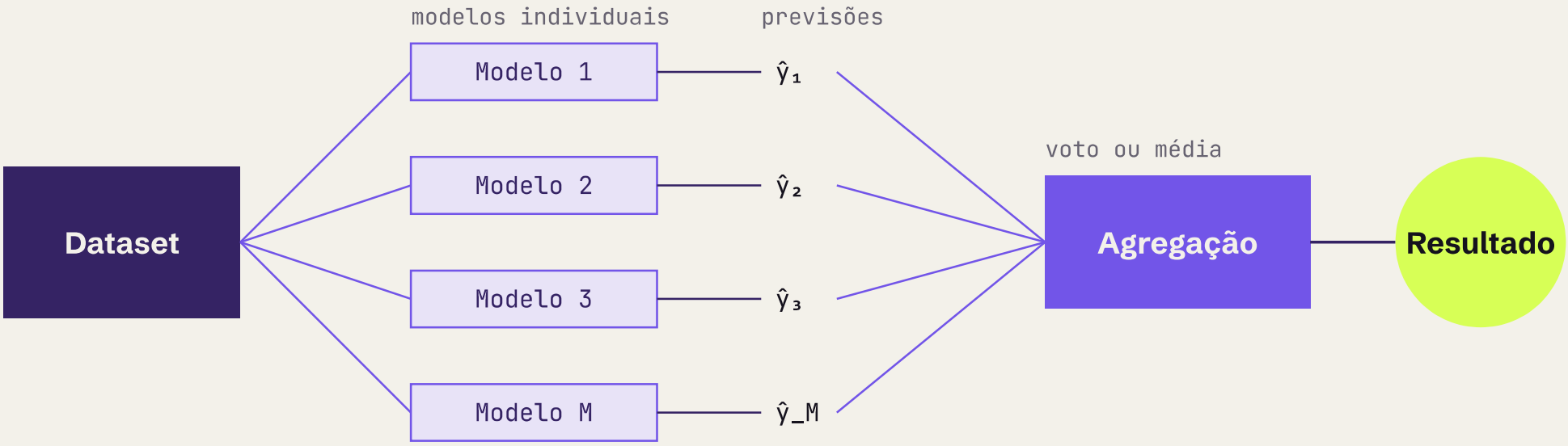
MANY MODELS. ONE ANSWER.

Ensemble e Bagging

Combinar modelos, e quando isso realmente ajuda.

Ensemble Learning

Um conjunto de modelos cujas previsões são combinadas para produzir uma previsão final.



Quando combinar ajuda

3 classificadores, cada um com 70% de acerto, votação majoritária.

Erros independentes

$P(3 \text{ acertos}) = 0{,}7^3 = 0{,}343$

$P(2 \text{ acertos}) = 3 \cdot 0{,}7^2 \cdot 0{,}3 = 0{,}441$

$P(\text{maioria acerta}) = 0{,}343 + 0{,}441$

78,4% +8,4 p.p.

Modelos idênticos

Os três erram nos mesmos pontos

A votação repete a resposta de cada um

Nenhuma informação nova

70,0% sem ganho

O ganho depende da qualidade de cada modelo e da diversidade dos seus erros. Na prática, os erros são parcialmente correlacionados.

BOOTSTRAP. REPEAT. AGGREGATE.

Bootstrap Aggregating

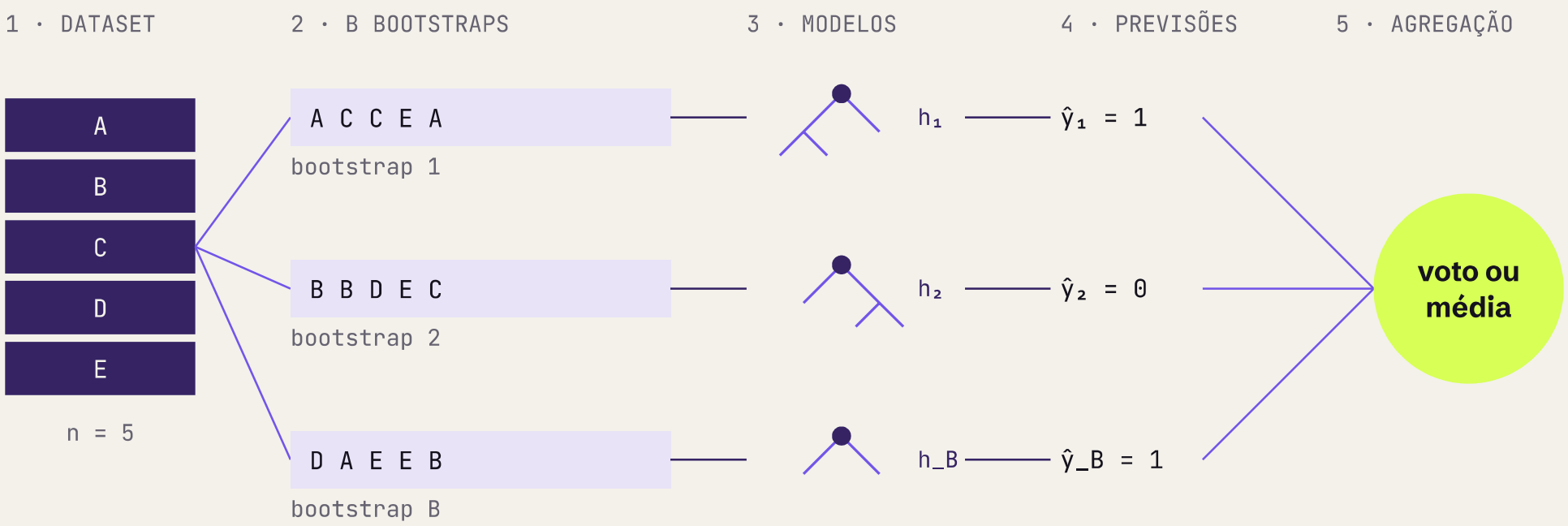
Bootstrap

Criar amostras de treino por amostragem com reposição.

Aggregating

Combinar as previsões dos modelos treinados nessas amostras.

Pipeline do Bagging

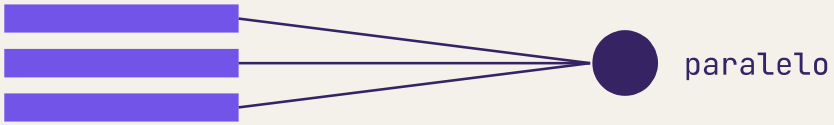


n = nº de amostras do dataset **B** = nº de bootstraps = nº de modelos **h_b** = modelo treinado no bootstrap b

Bagging e Boosting

Bagging

HOJE · 7.3



Modelos independentes

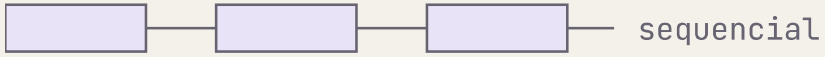
Cada um em uma amostra bootstrap

Votação ou média simples

Foco: reduzir variância

Boosting

PRÓXIMA · 7.4



Modelos em sequência

Cada um corrige erros dos anteriores

Combinação geralmente ponderada

Foco: reduzir viés

03

SAMPLE. REPLACE. REPEAT.

Bootstrap e Out-of-Bag

Amostragem com reposição e as amostras que ficam de fora.

Amostragem sem e com reposição

Sem reposição



O elemento sorteado sai da urna

Nenhuma repetição possível

5 de 5 = sempre os mesmos elementos

Com reposição

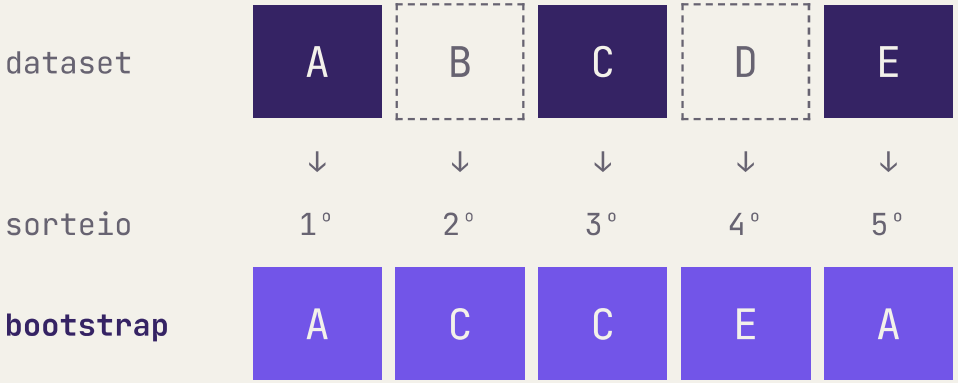


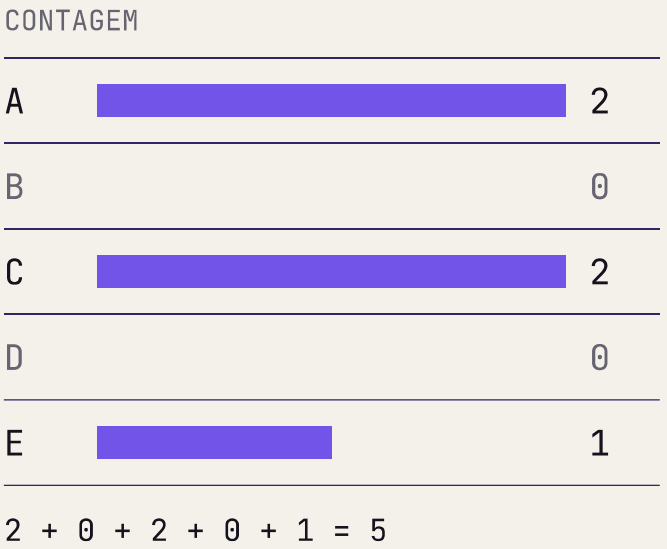
O elemento volta à urna após cada sorteio

Cada sorteio: chance 1/n para todos

Repetições e ausências acontecem

Uma amostra bootstrap, sorteio a sorteio





5
seleções

3
amostras distintas

2
nunca sorteadas

Bootstrap com NumPy

```python
import numpy as np

dados = np.array(["A", "B", "C", "D", "E"])

rng = np.random.default_rng(42)

indices = rng.choice(
    len(dados),
    size=len(dados),
    replace=True
)

amostra = dados[indices]

print("Índices:", indices)
print("Bootstrap:", amostra)
```

`default_rng(42)`
Gerador aleatório; 42 é a semente que torna os sorteios reproduzíveis

`choice(len(dados), …)`
Sorteia inteiros entre 0 e 4: os índices

`size=len(dados)`
5 sorteios, o tamanho do dataset

`replace=True`
Com reposição: repetições permitidas

`dados[indices]`
Indexação por array: devolve as letras sorteadas

Lendo a saída do bootstrap

SAÍDA · seed = 42

```
Índices: [ executar no notebook ]
Bootstrap: [ executar no notebook ]
```

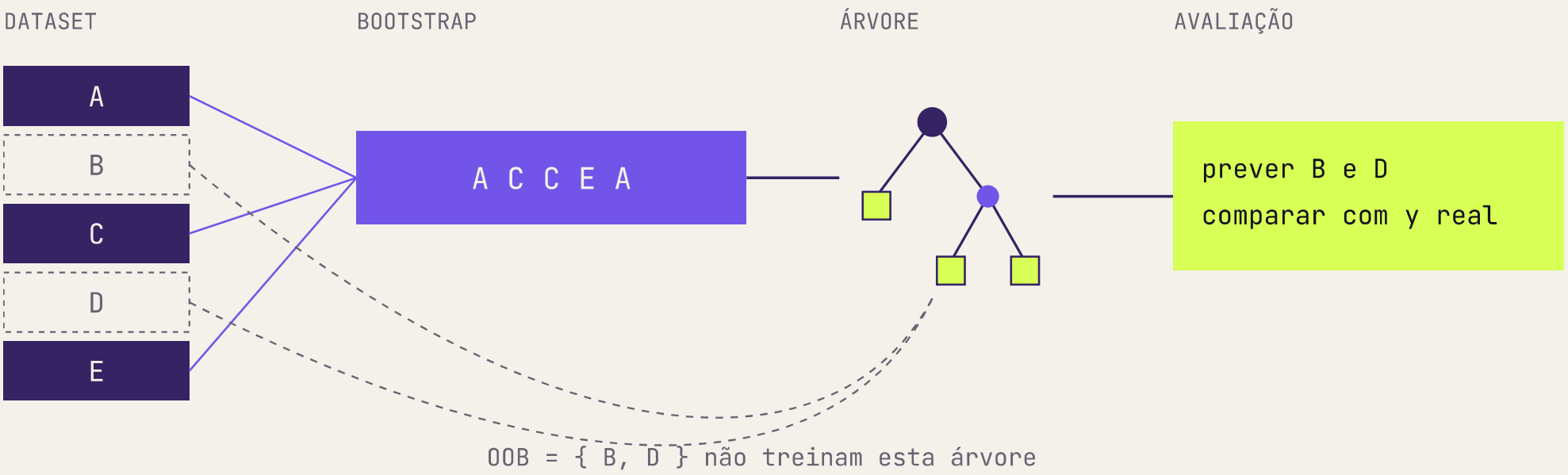
Resultado pendente de execução. Os índices não são reproduzidos aqui para não apresentar valores não calculados.

ROTEIRO DE LEITURA

1. Quais índices se repetiram?
2. Quantas amostras distintas apareceram?
3. Quais ficaram de fora? Estas são OOB.
4. Troque a semente: o tamanho muda?

```
oob = np.setdiff1d(np.arange(len(dados)), indices)
print("OOB:", dados[oob])
```

Out-of-Bag: as amostras que ficaram de fora



OOB é relativo a cada árvore: B está fora deste bootstrap, mas pode estar dentro de outro.

Probabilidade de uma amostra ser OOB

1	Sortear a amostra i em 1 sorteio	$1/n$
2	Não sortear i em 1 sorteio	$P = 1 - 1/n$
3	Não sortear i em n sorteios independentes	$(1 - 1/n)^n$

- n — nº de amostras do dataset = nº de sorteios
- i — uma amostra específica qualquer
- P — probabilidade de i não sair em um sorteio

Exemplo · n = 5

$1 - 1/5 = 0,8$

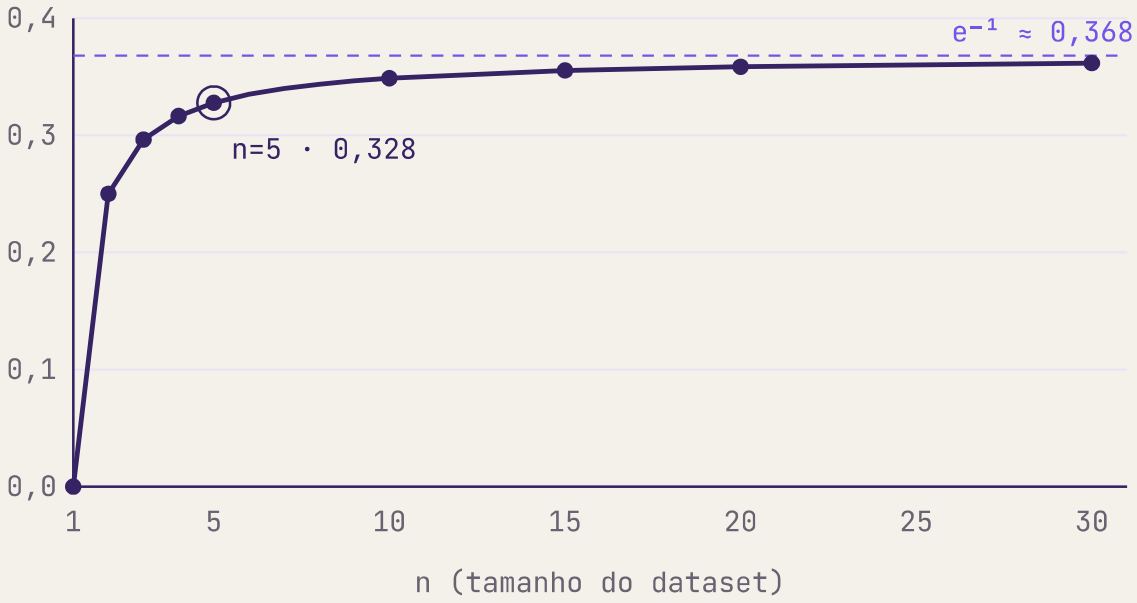
$0,8^5 = 0,32768$

$E[nº\ OOB] = 5 \cdot 0,32768 \approx 1,64$

32,8%

No exemplo A C C E A saíram 2 OOB. 1,64 é a média; cada bootstrap pode ter de 0 a 4.

Para n grande, P(OOB) ≈ e^{-1}



n	$(1 - 1/n)^n$
5	0,3277
10	0,3487
100	0,3660
450	0,3675
1000	0,3677
n → ∞	$e^{-1} \approx 0,3679$

≈ 36,8% ficam de fora e ≈ 63,2% aparecem ao menos uma vez, **em média**. Cada bootstrap varia.

04

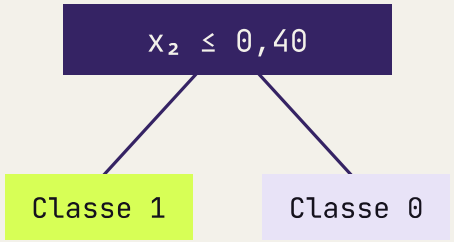
MANY TREES. ONE VOTE.

Agregação e diversidade

Votação, média e por que árvores diferentes importam.

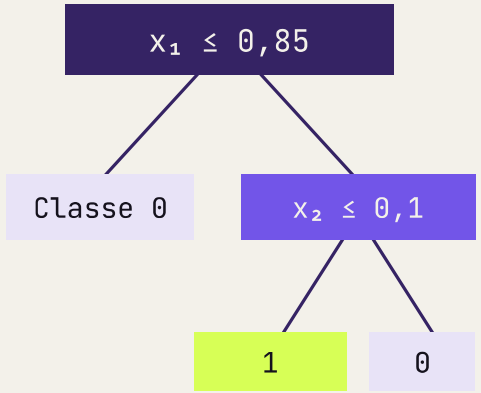
Três bootstraps, três árvores

A C C E A



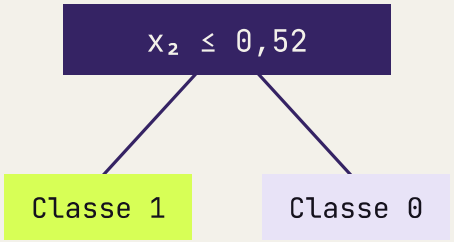
Árvore 1 · prof. 1 → **Classe 1**

B B D E C



Árvore 2 · prof. 2 → **Classe 0**

D A E E B



Árvore 3 · prof. 1 → **Classe 1**

DECISÃO COLETIVA

1

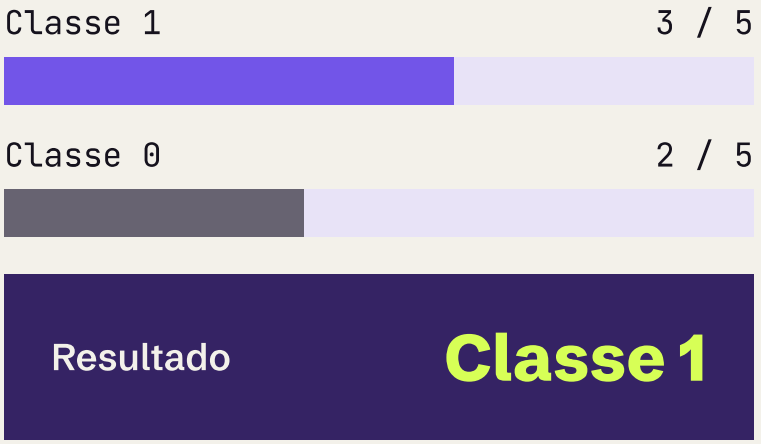
2 votos de 3

Estruturas ilustrativas. Variam: dados, perguntas, profundidade e previsões.

Votação majoritária

Árvore 1	Classe 1
Árvore 2	Classe 0
Árvore 3	Classe 1
Árvore 4	Classe 1
Árvore 5	Classe 0

$$\hat{y} = \text{argmax}_c \sum_b \mathbb{1}[\, h_b(x) = c \,]$$

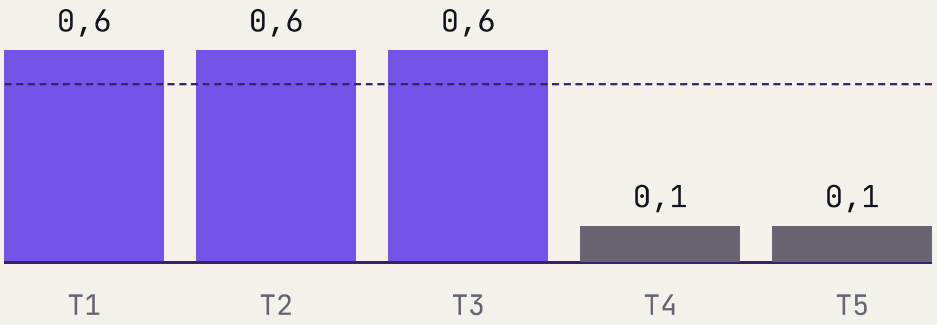


Empates: impossíveis com 2 classes e nº ímpar de árvores; com nº par, exigem regra de desempate.

Mais modelos: cada voto pesa menos; a proporção fica mais estável.

O Scikit-learn faz a média das probabilidades

P(classe 1 | x) estimada por árvore



Linha tracejada: limiar 0,5.

Votação rígida (rótulos)

3 votos × 2 votos → **Classe 1**

Média das probabilidades (sklearn)

(0,6+0,6+0,6+0,1+0,1)/5 = 0,4

0,4 < 0,5 → **Classe 0**

predict() escolhe a classe de maior probabilidade média. Com folhas puras os dois métodos coincidem com frequência, mas não obrigatoriamente.

Agregação em classificação e regressão

`RandomForestClassifier`

Classificação

Combina as previsões para escolher uma classe: votação no conceito, média de probabilidades no Scikit-learn.

$$\hat{y} = \text{argmax}_c \ (1/B) \ \Sigma_b \ \hat{p}_b(c \mid x)$$

max_features padrão: "sqrt"

`RandomForestRegressor`

Regressão

`T1 → 100` `T2 → 110` `T3 → 90`

`(100 + 110 + 90) / 3 =` **`100`**

$$\hat{y} = (1/B) \ \Sigma_b \ h_b(x)$$

max_features padrão: 1.0 (todas)

Variância da média e correlação entre árvores

$$\mathrm{Var} = \rho\sigma^2 + (1 - \rho)\sigma^2/B$$

- σ^2 variância da previsão de uma árvore
- ρ correlação entre previsões de duas árvores
- B número de árvores na média

2º termo cai quando B cresce

1º termo não depende de B: só cai se ρ cair

Supõe σ^2 e ρ iguais para todos os pares (Hastie, Tibshirani e Friedman, ESL §15.2).

$\sigma^2 = 1 \cdot B = 500$

Árvores parecidas · ρ = 0,9

0,9 + 0,1/500

≈ 0,900

Árvores diversas · ρ = 0,3

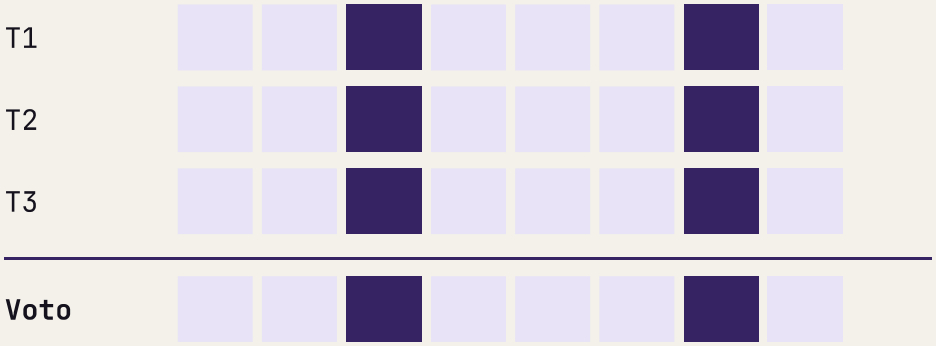
0,3 + 0,7/500

≈ 0,301

Erros correlacionados e erros diversos

Cenário A · árvores semelhantes

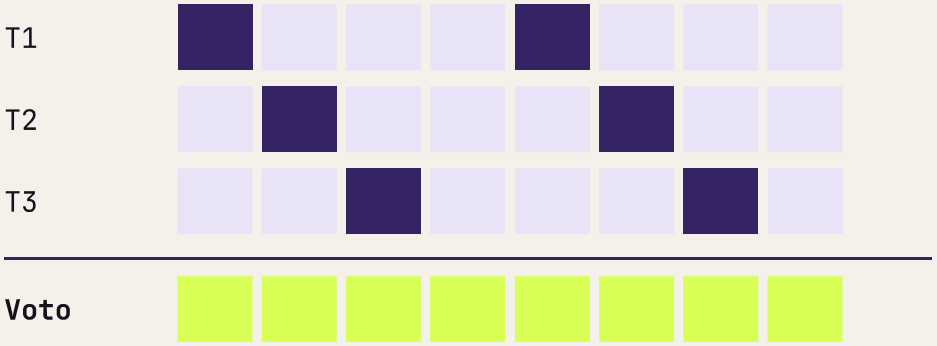
■ erro □ acerto · 8 pontos de teste



Cada árvore: 6/8. Votação: **6/8**. Sem ganho.

Cenário B · árvores diversas e úteis

■ erro □ acerto · 8 pontos de teste



Cada árvore: 6/8. Votação: **8/8**. Diversidade útil.

Exemplo construído. Diversidade ilimitada não garante melhora: árvores piores que o acaso pioram a votação.

05

500 TREES. ONE PREDICTION.

Random Forest

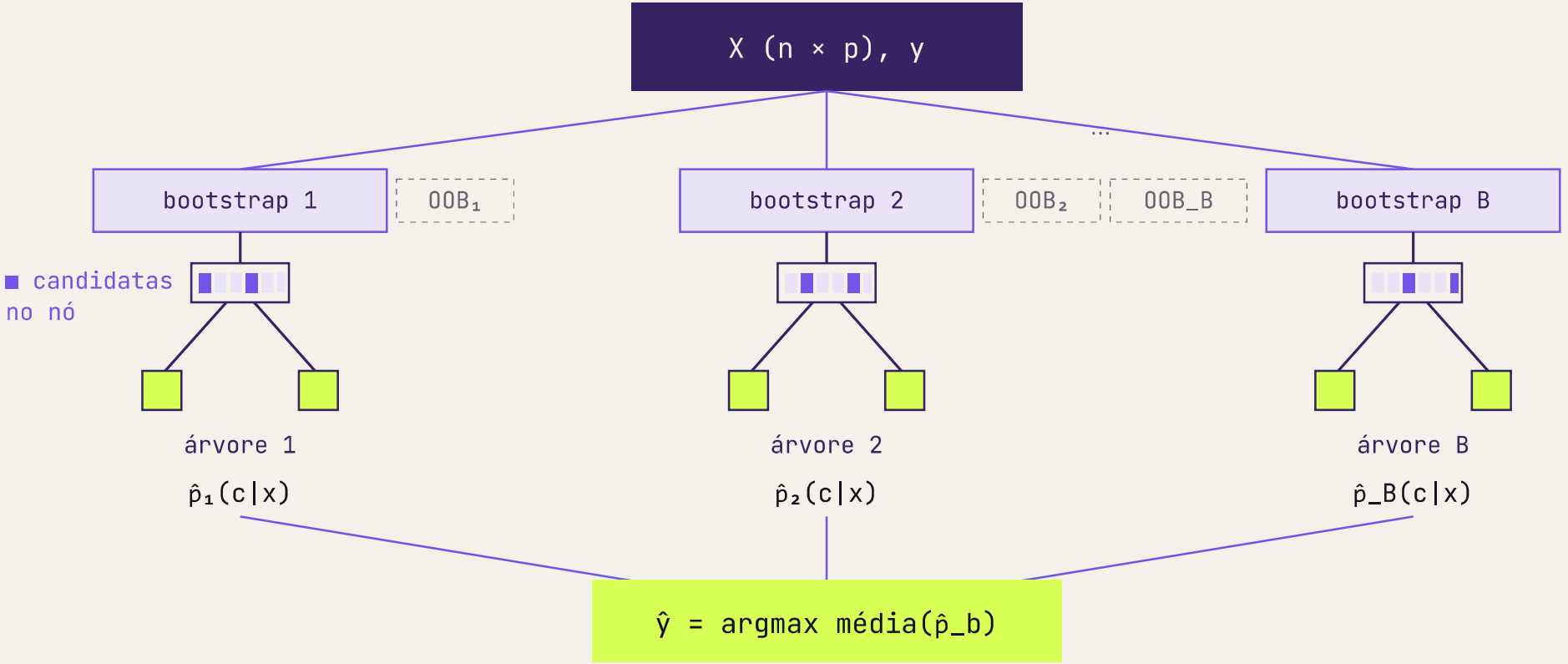
Algoritmo, características, hiperparâmetros, OOB e importância.

O que é uma Random Forest

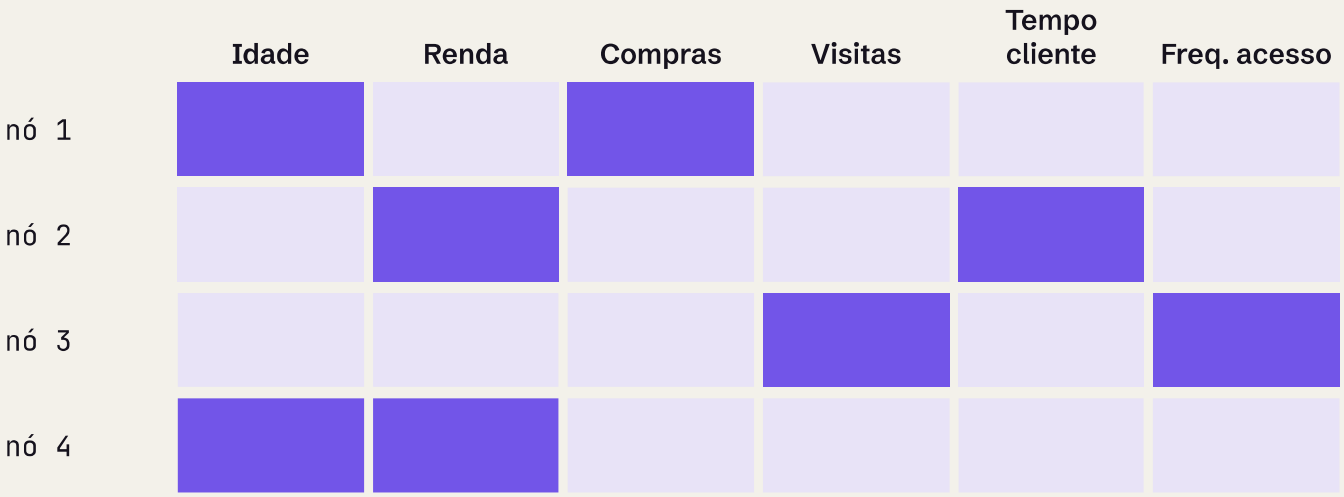
01	Criar B amostras bootstrap	bagging
02	Treinar uma árvore em cada amostra	bagging
03	**Em cada divisão, sortear um subconjunto de características**	NOVO
04	Produzir previsões individuais	bagging
05	Agregar as previsões	bagging

Ensemble de árvores com mecanismos de aleatoriedade para promover diversidade.

Funcionamento completo



Amostragem de características por divisão



■ candidatas sorteadas naquele nó · mesma árvore

```
max_features="sqrt"
```

p = 6

√6 ≈ 2,449

⌊2,449⌋ = **2** por divisão

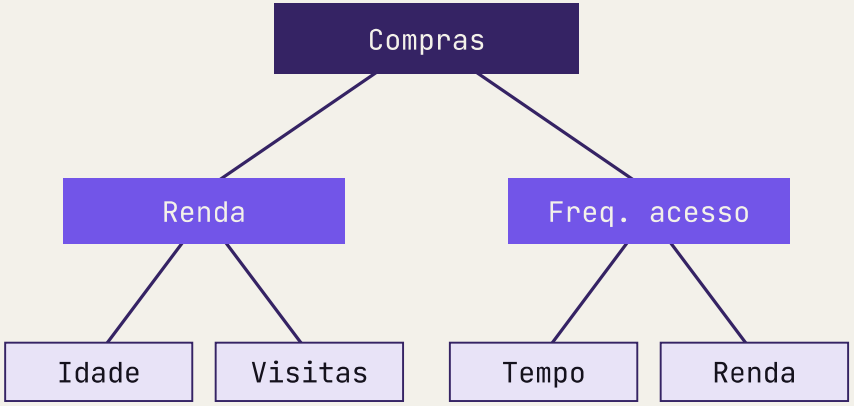
p = nº de características. Cada nó sorteia de novo as suas candidatas.

Se nenhuma divisão válida é encontrada entre as candidatas, o Scikit-learn continua avaliando outras características.

Sorteio por divisão, não por árvore

Por divisão

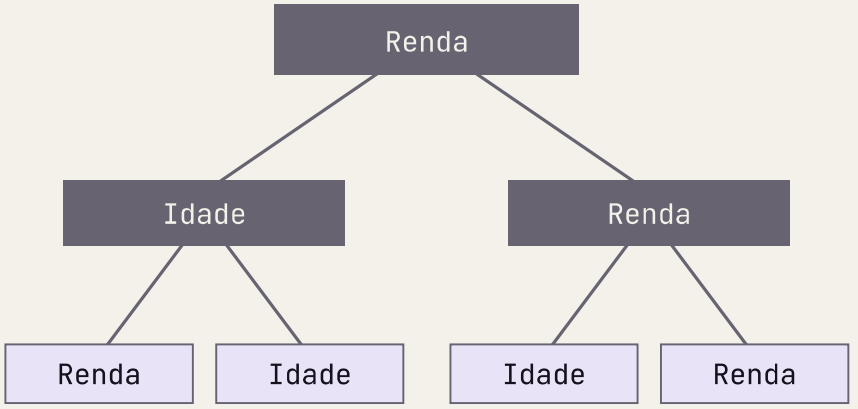
RANDOM FOREST



Novo sorteio em cada nó. A árvore pode usar todas as características ao longo da estrutura.

Fixo por árvore

OUTRO MÉTODO



Um único subconjunto (ex.: Idade, Renda) para a árvore inteira. Não é o que max_features faz.

Bagging de árvores e Random Forest

ASPECTO	BAGGING DE ÁRVORES	RANDOM FOREST
Amostras de treino	Bootstrap	Bootstrap (configurável)
Características por divisão	Todas	Subconjunto sorteado
Fontes de diversidade	Linhas	Linhas e colunas
Agregação	Voto ou média	Média de probabilidades / média
Classe no Scikit-learn	`BaggingClassifier`	`RandomForestClassifier`

RandomForestClassifier(max_features=None, bootstrap=True) equivale essencialmente a Bagging de árvores. O comportamento depende dos hiperparâmetros.

Hiperparâmetros: floresta e árvores

PARÂMETRO	PADRÃO	O QUE CONTROLA · EFEITO AO AUMENTAR
`n_estimators`	`100`	Nº de árvores (B). Média mais estável; mais custo; ganho decrescente.
`max_depth`	`None`	Profundidade máxima. Árvores mais complexas; menor viés, maior variância.
`max_features`	`"sqrt"`	Candidatas por divisão. Mais candidatas: árvores mais fortes e mais correlacionadas.
`min_samples_split`	2	Mínimo para dividir um nó. Maior: árvores mais rasas e suaves.
`min_samples_leaf`	1	Mínimo por folha. Maior: menos folhas minúsculas, menos ruído decorado.

Padrões do RandomForestClassifier no Scikit-learn 1.x. Confira na documentação da versão instalada.

Hiperparâmetros: amostragem e execução

bootstrap — True

Liga a amostragem bootstrap. False: cada árvore usa todo o treino.

oob_score — False

Calcula a estimativa OOB no fit. Exige bootstrap=True, senão ValueError.

random_state — None

Semente dos bootstraps e dos sorteios de características. Garante reprodutibilidade.

n_jobs — None

Nº de núcleos em paralelo. –1 = todos. Muda o tempo, não o resultado.

Alterando parâmetros

```python
# 1 · padrão
rf_padrao = RandomForestClassifier(random_state=42)

# 2 · árvores mais simples
rf_simples = RandomForestClassifier(
    max_depth=4,
    min_samples_leaf=5,
    random_state=42
)

# 3 · sem sorteio de características = bagging
rf_bagging = RandomForestClassifier(
    max_features=None,
    random_state=42
)
```

EFEITO ESPERADO

rf_simples

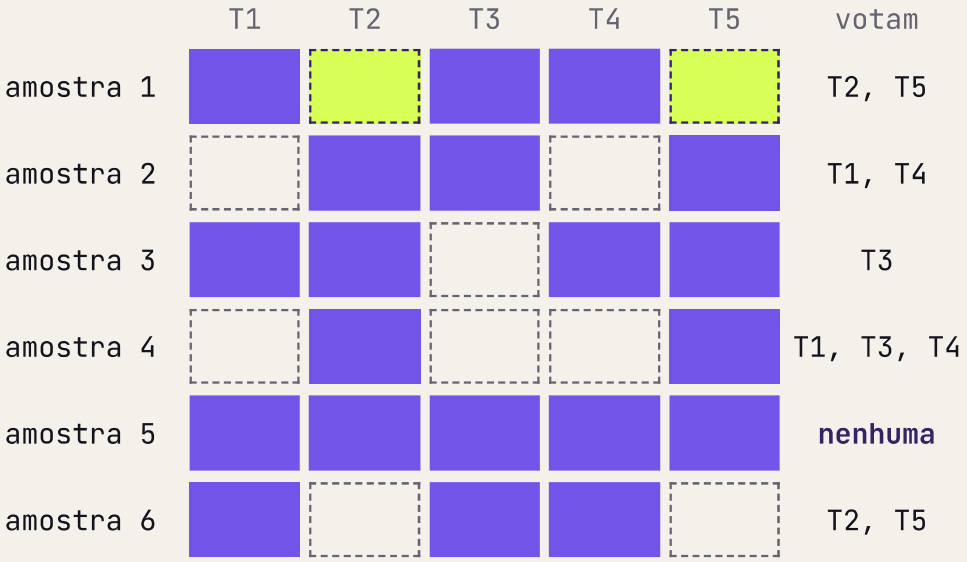
Treino menor, fronteira mais suave. Teste pode subir ou cair.

rf_bagging

Árvores mais correlacionadas. Com só 2 características, a diferença pode ser pequena.

Sem números aqui: o desempenho é medido no exercício (etapas 17 e 18).

Out-of-Bag como estimativa de desempenho



1	Encontrar as árvores em que a amostra foi OOB
2	Prever a amostra só com essas árvores
3	Agregar (média das probabilidades)
4	Comparar com o rótulo verdadeiro

oob_score_ = acurácia dessas previsões sobre o treino.

Ativando e lendo o OOB

```python
modelo = RandomForestClassifier(
    n_estimators=500,
    bootstrap=True,   # obrigatório
    oob_score=True,
    random_state=42
)
modelo.fit(X_train, y_train)

print(modelo.oob_score_)
```

```
# bootstrap=False + oob_score=True
ValueError: Out of bag estimation only
available if bootstrap=True
```

CUIDADOS

Usa o treino de forma específica: cada amostra julgada por árvores que não a viram

É uma estimativa interna, calculada no fit

Não equivale à acurácia no teste

Poucas árvores: amostras sem cobertura OOB (UserWarning)

Não substitui a avaliação final independente

Importância por redução de impureza

```
modelo.feature_importances_
```

$$\text{Imp}(j) \propto (1/B) \sum_b \sum_{t \text{ usa } j} (N_t/N) \cdot \Delta G_t$$

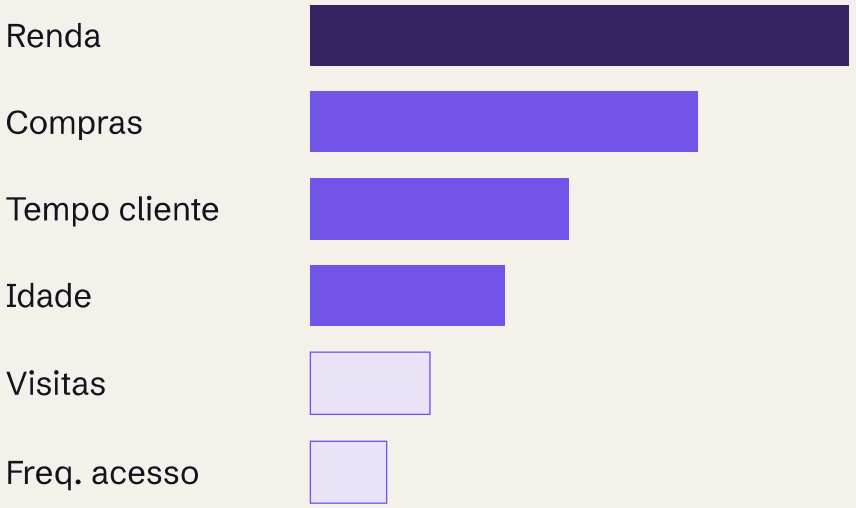
ΔG_t	redução de Gini no nó t
N_t/N	fração das amostras que chegam a t
$\Sigma = 1$	normalizada: as importâncias somam 1

Muitos valores possíveis → mais cortes → importância inflada

Correlacionadas → importância dividida, difícil de ler

Não é causalidade → mostra uso pelo modelo

FORMATO · valores ilustrativos



Impureza e permutação

1 · MEDIR

x_1	x_2	y
0,8	1,2	1
-0,3	0,4	0
1,6	-0,5	1

Acurácia de referência

2 · EMBARALHAR x_1

x_1 ↻	x_2	y
1,6	1,2	1
0,8	0,4	0
-0,3	-0,5	1

Relação x_1 ↔ y quebrada

3 · MEDIR DE NOVO

`queda = acc_ref - acc_perm`

Queda grande → modelo dependia de x_1

4 · REPETIR

`n_repeats=10`

Média e desvio das quedas

	IMPUREZA (MDI)	PERMUTAÇÃO
Dados	Treino, durante o fit	Qualquer conjunto (ex.: teste)
Mede	Redução de Gini acumulada	Queda na métrica
Custo · viés	Rápida · infla muitos valores	Mais lenta · sofre com correlação

ONE TREE VS. 500.

Árvore de Decisão

Uma única estrutura de regras

Fácil de interpretar quando pequena

Pode apresentar alta variância

Treino e previsão rápidos

Random Forest

Conjunto de estruturas + agregação

Interpretação global mais difícil

Pode reduzir a variância

Custo computacional geralmente maior

Nem sempre a floresta terá acurácia superior. Depende do dataset.

Overfitting e generalização

Erro de treino	Nos dados usados no fit. Otimista.
Erro de validação	Dados separados para escolher hiperparâmetros.
Erro de teste	Medido uma vez, no final. Estima a generalização.
Variância	Quanto o modelo muda com outros dados de treino.
Generalização	Desempenho em dados novos.



Usar apenas a figura calculada no notebook. Nenhuma curva fictícia.

Quiz rápido

01 O que é Ensemble Learning?

02 O que significa Bagging?

03 O que é Bootstrap?

04 O que significa amostragem com reposição?

05 Por que algumas amostras aparecem repetidas?

06 O que são amostras OOB?

07 Por que treinamos várias árvores?

08 O que significa diversidade entre modelos?

Quiz rápido

09 Qual a diferença entre Bagging e Random Forest?

10 Para que serve `max_features`?

11 O que controla `n_estimators`?

12 O que significa `oob_score_`?

13 Por que 500 árvores não garantem acurácia superior?

14 O que representa a importância de uma característica?

15 Por que uma Random Forest é menos interpretável que uma árvore pequena?

06

ONE TREE VS. 500.

Uma árvore contra 500 árvores

Aula_7_3_Exercicio_Guiado_Random_Forest.ipynb

Vinte etapas, do notebook vazio ao resultado

01 Imports	02 Criar dataset	03 Visualizar	04 Treino e teste	05 Treinar árvore
06 Prever (árvore)	07 Avaliar árvore	08 Região de decisão	09 Criar floresta	10 Treinar floresta
11 Prever (floresta)	12 Avaliar floresta	13 Fronteiras lado a lado	14 OOB	15 Importância MDI
16 Permutação	17 Nº de árvores	18 Profundidade	19 Tabela final	20 Interpretação

■ núcleo em sala ■ continuação guiada / extensão ■ fechamento

Imports

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.inspection import permutation_importance
```

```
# auxiliares: custo computacional e tabela finalimport time
import pandas as pd
```

numpy arrays e grade

pyplot gráficos

make_moons dataset sintético

train_test_split separar treino e teste

DecisionTree… uma árvore

RandomForest… a floresta

accuracy_score fração de acertos

confusion_matrix acertos e erros por classe

permutation_… importância por permutação

time · pandas custo e tabela

Criar e visualizar o dataset

```
X, y = make_moons(
    n_samples=600,
    noise=0.28,
    random_state=42
)

print("Formato de X:", X.shape)
print("Formato de y:", y.shape)
print("Classes:", np.unique(y))
print("Por classe:", np.bincount(y))
```

```
Formato de X: (600, 2)
Formato de y: (600,)
Classes: [0 1]
Por classe: [300 300]
```



600 amostras · 2 características (x_1, x_2) · 2 classes

Separar treino e teste

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

print(X_train.shape, X_test.shape)
print(np.bincount(y_train), np.bincount(y_test))
```

```
(450, 2) (150, 2)
[225 225] [75 75]
```



test_size=0.25 25% para teste

random_state=42 mesmo split para todos

stratify=y mantém 50/50 nas classes

Um único split para todos os modelos. Bootstrap acontece depois, dentro do treino.

Treinar, prever e avaliar uma árvore

```python
# Etapa 5 · treinar
arvore = DecisionTreeClassifier(random_state=42)
arvore.fit(X_train, y_train)
print("Profundidade:", arvore.get_depth())
print("Folhas:", arvore.get_n_leaves())

# Etapa 6 · prever
pred_arvore = arvore.predict(X_test)
print("Previsto:", pred_arvore[:10])
print("Real:    ", y_test[:10])

# Etapa 7 · avaliar
acc_treino_arvore = accuracy_score(y_train, arvore.predict(X_train))
acc_teste_arvore = accuracy_score(y_test, pred_arvore)
cm_arvore = confusion_matrix(y_test, pred_arvore)
```

SAÍDA

```
Profundidade: —
Folhas: —
Acc. treino: —
Acc. teste: —
```

Preencher com a saída do Run All.

	prev 0	prev 1
real 0	VN	FP
real 1	FN	VP

Função de região de decisão

```
COR_CLASSE = ["#352364", "#D7FF56"]
COR_REGIAO = ["#C9BEF5", "#F1FFC2"]
LIMITES = (X[:, 0].min() - 0.5, X[:, 0].max() + 0.5,
           X[:, 1].min() - 0.5, X[:, 1].max() + 0.5)

def plotar_fronteira(modelo, X_plot, y_plot, ax, titulo, resolucao=300):
    x_min, x_max, y_min, y_max = LIMITES
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, resolucao),
                         np.linspace(y_min, y_max, resolucao))
    grade = np.c_[xx.ravel(), yy.ravel()]          # (90000, 2)
    Z = modelo.predict(grade).reshape(xx.shape)    # (300, 300)
    ax.contourf(xx, yy, Z, levels=[-0.5, 0.5, 1.5], colors=COR_REGIAO)
    for classe in (0, 1):
        m = y_plot == classe
        ax.scatter(X_plot[m, 0], X_plot[m, 1], s=18, c=COR_CLASSE[classe],
                   edgecolors="#15121D", linewidths=0.4, label=f"Classe {classe}")
    ax.set_xlim(x_min, x_max); ax.set_ylim(y_min, y_max)
    ax.set_title(titulo); ax.set_xlabel("x1"); ax.set_ylabel("x2")
    ax.legend(loc="upper right")
```

A fronteira aprendida pela árvore

```
fig, ax = plt.subplots(figsize=(7, 5.5))
plotar_fronteira(arvore, X_test, y_test,
    ax, "Árvore de Decisão")
plt.show()
```

meshgrid grade 300 × 300

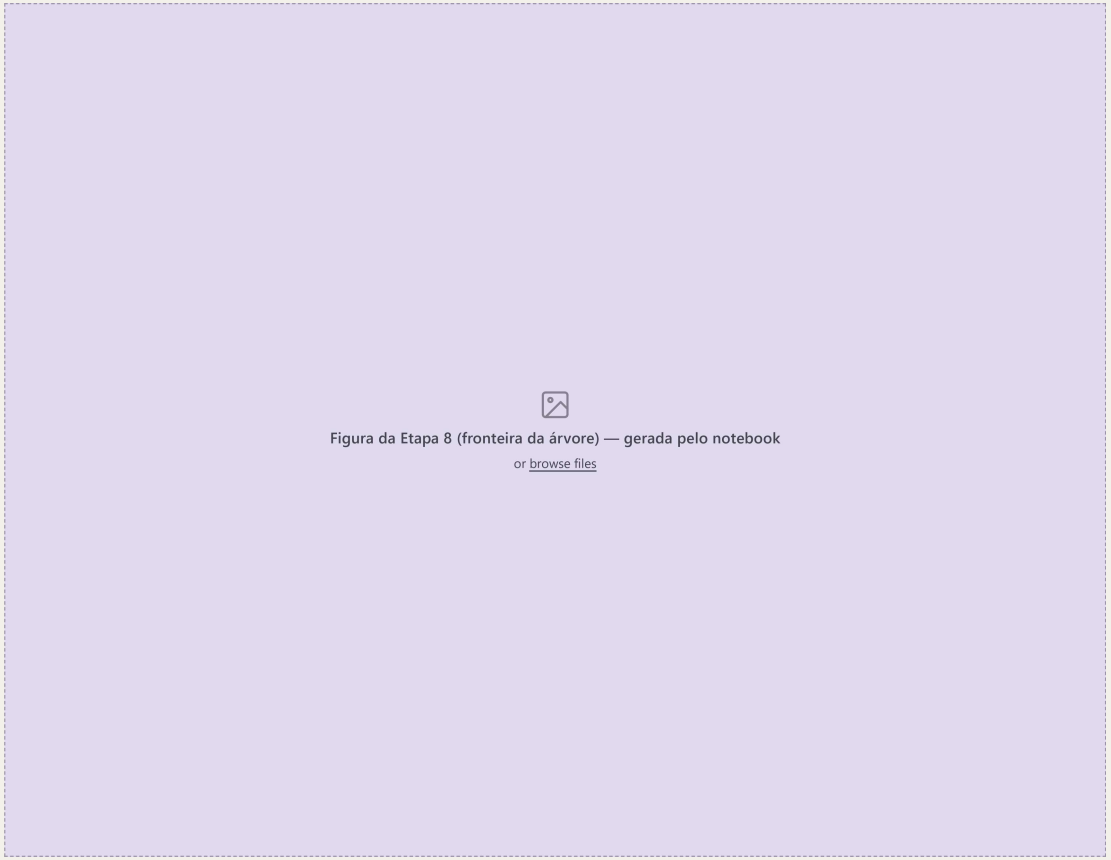
predict classe de cada ponto

reshape volta à forma da grade

contourf pinta as regiões

scatter desenha as amostras

Esperado: fronteira em degraus, paralela aos eixos, com possíveis ilhas.



Criar, treinar, prever e avaliar a floresta

```
# Etapa 9 · criar
floresta = RandomForestClassifier(
    n_estimators=500,
    bootstrap=True,
    oob_score=True,
    random_state=42,
    n_jobs=-1
)
# Etapa 10 · treinar 500 árvores
t0 = time.perf_counter()
floresta.fit(X_train, y_train)
tempo_floresta = time.perf_counter() - t0
# Etapa 11 · prever
pred_floresta = floresta.predict(X_test)
print("Discordâncias:", (pred_floresta ≠ pred_arvore).sum())
# Etapa 12 · avaliar
acc_treino_floresta = accuracy_score(y_train, floresta.predict(X_train))
acc_teste_floresta = accuracy_score(y_test, pred_floresta)
cm_floresta = confusion_matrix(y_test, pred_floresta)
```

500 árvores (B)

bootstrap 450 sorteios por árvore

oob_score estimativa interna

random_state sorteios reproduzíveis

n_jobs=-1 todos os núcleos

---

max_features padrão "sqrt": ⌊√2⌋ = 1 candidata por divisão.

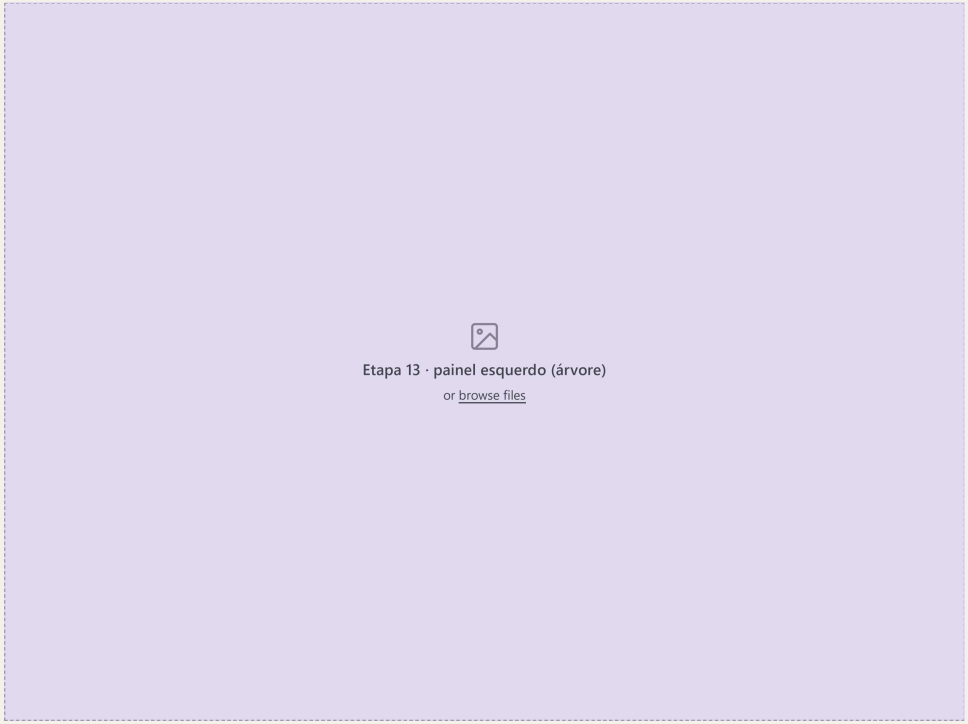
```
Acc. treino: —
Acc. teste: —
Tempo (s): —
```

ONE TREE VS. 500.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5.5), sharex=True, sharey=True)
```

Árvore de Decisão

1 árvore



Random Forest

500 árvores



Mesmos limites, escala, cores e amostras de teste. Figuras geradas pelos modelos treinados.

OOB e acurácia de teste

```
print("OOB Score:", floresta.oob_score_)
print("Acc. teste:", acc_teste_floresta)
print("Diferença:",
      floresta.oob_score_ - acc_teste_floresta)
```

OOB

TESTE

Preencher com a saída do Run All.

	OOB	TESTE
Dados	450 de treino	150 novas
Árvores	Só as que não viram a amostra (~36,8%)	Todas as 500
Uso	Ajuste interno	Avaliação final

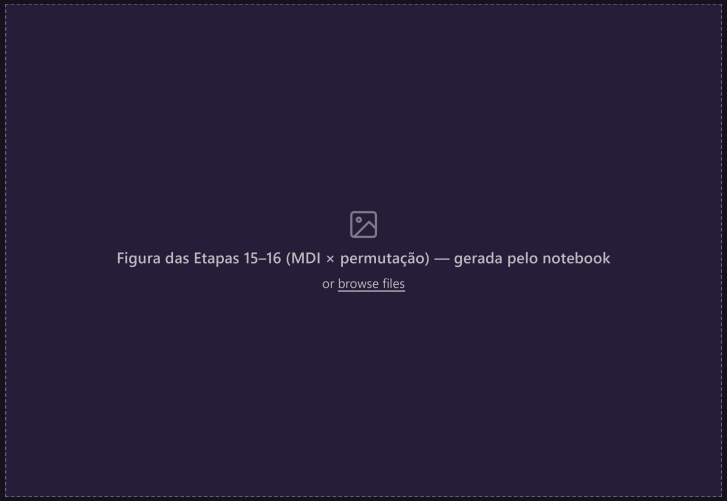
No teste, 1 erro = 1/150 ≈ 0,67 p.p. Diferenças pequenas entre OOB e teste são esperadas.

Importância: impureza e permutação

```
# Etapa 15 · impureza (MDI)
nomes = ["x1", "x2"]
imp_mdi = floresta.feature_importances_
print(dict(zip(nomes, imp_mdi)), imp_mdi.sum())

# Etapa 16 · permutação no teste
perm = permutation_importance(
    floresta,
    X_test,
    y_test,
    n_repeats=10,
    random_state=42
)
print(perm.importances_mean, perm.importances_std)

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].bar(nomes, imp_mdi, color="#7255E8")
axes[1].bar(nomes, perm.importances_mean,
            yerr=perm.importances_std, color="#352364")
```



MDI: soma 1, calculada no treino. Permutação: queda de acurácia no teste, média ± desvio de 10 repetições.

Quantidade de árvores

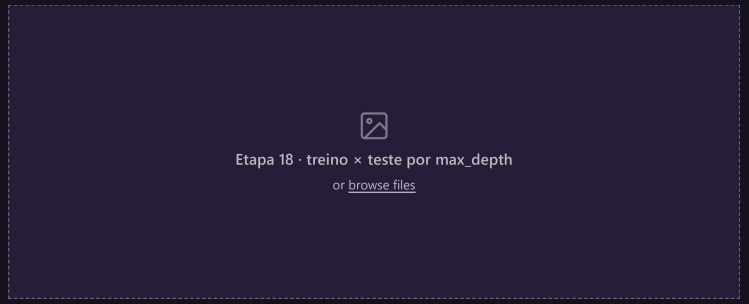
```
quantidades = [1, 5, 10, 25, 50, 100, 200, 500]
resultados_n = []
for n in quantidades:
    usa_oob = n ⩾ 25   # cobertura OOB suficiente
    rf_n = RandomForestClassifier(n_estimators=n,
        oob_score=usa_oob, random_state=42, n_jobs=-1)
    rf_n.fit(X_train, y_train)
    resultados_n.append({
        "n_estimators": n,
        "acc_treino": accuracy_score(y_train, rf_n.predict(X_train)),
        "acc_teste": accuracy_score(y_test, rf_n.predict(X_test)),
        "oob": rf_n.oob_score_ if usa_oob else np.nan,
    })
tabela_n = pd.DataFrame(resultados_n)
```

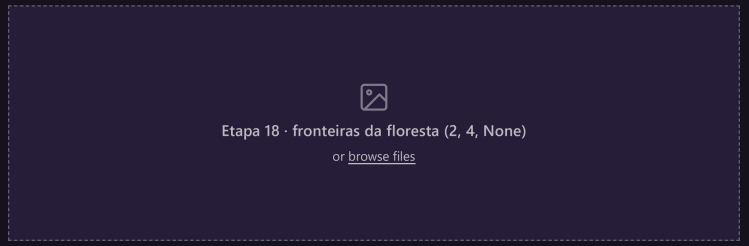


Mesmo split. A acurácia de teste não precisa crescer de forma monotônica.

Profundidade das árvores

```
profundidades = [1, 2, 3, 4, 6, 8, 10, None]
resultados_d = []
for d in profundidades:
    a = DecisionTreeClassifier(max_depth=d,
            random_state=42).fit(X_train, y_train)
    f = RandomForestClassifier(n_estimators=500,
            max_depth=d, random_state=42,
            n_jobs=-1).fit(X_train, y_train)
    for nome, m in [("Árvore", a), ("Floresta", f)]:
        resultados_d.append({"modelo": nome,
            "max_depth": str(d),
            "acc_treino": m.score(X_train, y_train),
            "acc_teste": m.score(X_test, y_test)})
tabela_d = pd.DataFrame(resultados_d)
```





model.score() em classificação = acurácia.

Comparação final

MODELO	ÁRVORES	MAX_DEPTH	TREINO	TESTE	MATRIZ (TESTE)	OOB
Árvore de Decisão	1	None	—	—	[[—, —], [—, —]]	n/a
Árvore rasa	1	4	—	—	[[—, —], [—, —]]	n/a
Random Forest	500	None	—	—	[[—, —], [—, —]]	—
Random Forest rasa	500	4	—	—	[[—, —], [—, —]]	—

— = preencher com a saída do notebook (tabela_final). Nenhum valor estimado.

`print(tabela_final.to_string())`

Interpretação dos resultados

01 A floresta superou a árvore profunda no teste? Por quantos pontos?

02 A diferença foi pequena? 1 ponto = 0,67 p.p.

03 A árvore rasa foi suficiente?

04 Como as fronteiras se diferenciam?

05 O OOB ficou próximo do teste? O que ele informa?

06 Quais características foram importantes? MDI e permutação concordam?

07 Qual foi o custo de 500 árvores em tempo de treino e de previsão?

Conclusões a partir dos números calculados, não antes deles.

07

DEBUG. CHALLENGE. READ.

Erros comuns e desafio

Erro, correção, investigação e leitura.

Erros comuns e correções

#	ERRO	CORREÇÃO
01	Confundir Bagging e Boosting	Bagging: paralelo e independente. Boosting: sequencial e corretivo.
02	Bootstrap = divisão treino/teste	Split: uma vez, para avaliar. Bootstrap: dentro do treino, por árvore.
03	`rng.choice(n, n, replace=False)`	`rng.choice(n, n, replace=True)`
04	Todas as árvores veem os mesmos dados	Cada uma recebe um bootstrap diferente (~63,2% distintas, em média).
05	Amostrar características = amostrar linhas	Bootstrap sorteia linhas; max_features sorteia colunas a cada divisão.
06	`RandomForestClassifier()`	`RandomForestClassifier(random_state=42)`
07	Treino ≈ 100% = modelo bom	Generalização se mede no teste (e estima-se no OOB).

Erros comuns e correções

#	ERRO	CORREÇÃO
08	OOB = acurácia de teste	OOB: estimativa interna no treino. Teste: avaliação independente.
09	`bootstrap=False, oob_score=True`	`bootstrap=True, oob_score=True`
10	Mais árvores sempre aumentam a acurácia	Ganho decrescente; o teste pode oscilar.
11	Importância = causalidade	Importância mede o uso pelo modelo.
12	sklearn faz votação de rótulos	predict = argmax da média de predict_proba.
13	Comparar modelos com splits diferentes	Um único train_test_split com random_state fixo.
14	Chamar plotar_fronteira antes do def	Definir antes de usar. Testar com Restart → Run All.

INVESTIGANDO A RANDOM FOREST

01 Criar ou carregar outro dataset
02 Explorar as características
03 Separar treino e teste
04 Treinar uma árvore
05 Treinar uma Random Forest
06 Comparar quantidades de árvores
07 Investigar profundidades
08 Utilizar OOB
09 Examinar importâncias
10 Comparar métricas
11 Criar gráficos
12 Interpretar os resultados

ENTREGAS

Notebook executável
Código organizado
Treino e teste
Matrizes de confusão
Gráficos comparativos
Análise do OOB
Características importantes
Conclusão

`Desafio_TODO.ipynb` · `Desafio_Gabarito.ipynb`

Leitura complementar

2001	Breiman, L. **Random Forests**. Machine Learning, 45(1), 5–32.
1996	Breiman, L. **Bagging Predictors**. Machine Learning, 24(2), 123–140.
ISL	James, Witten, Hastie, Tibshirani, Taylor. **An Introduction to Statistical Learning with Applications in Python**, cap. 8.
ESL	Hastie, Tibshirani, Friedman. **The Elements of Statistical Learning**, 2ª ed., §8.7 e cap. 15.
DOCS	Scikit-learn User Guide: **Ensembles** e **Permutation feature importance**.
2007	Strobl, Boulesteix, Zeileis, Hothorn. **Bias in random forest variable importance measures**. BMC Bioinformatics, 8, 25.

ONE DECISION. HUNDREDS OF PERSPECTIVES.

Árvore → Ensemble → Bootstrap → Bagging → Diversidade → Random Forest → Agregação → OOB → Importância